

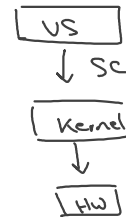
Why we need to understand device in Embedded Linux?

Driver ← device
Initialize exist

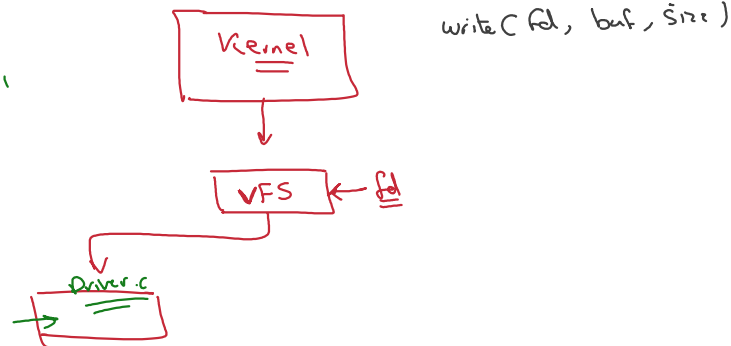
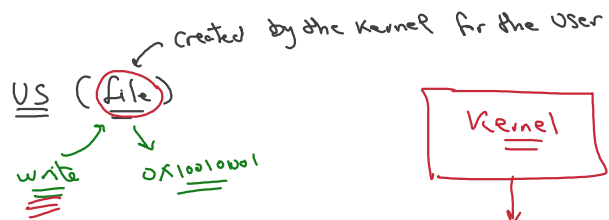
Linux

Userspace → doesn't have access HW

Kernel → access HW



eg: UART-CR → configure UART



Linux

Driver ← Device
UART UART



```

fady@android:/sys/class/leds/input3:capslock$ ls
brightness device max_brightness power subsystem

```

Linux
led.c
 void gpio_writch (①)
 {
 }
 }

- ① In order the Driver to init, we need the device .
- ② any functionality in driver will be accessed by a file

UART-SendBuffer



SendBuffer (hello)

Echo hello > SendBuffer

VS → (file)

VSS

(driver)

main {
 {
 , port-init {
 }
 }
 }

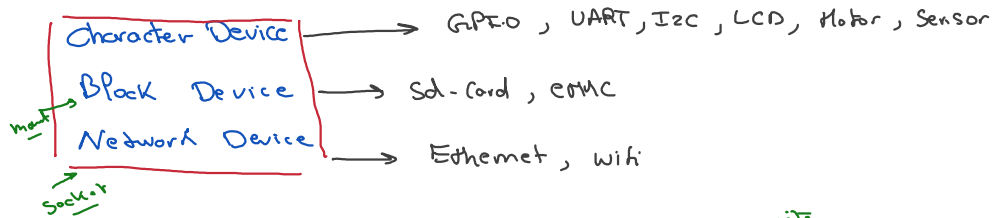


← open ty

(fd)

→ dprint ((fd) hello);

Driver-VSS



we will focus on character device

write
read
seek

Streaming of Bytes

[①] → Linux → vfs → Driver

Advantage

↳ speed

Disadvantage

↳ crash system



process file system



① Virtual Machine

② RPI

Makefile

↓
Build

Kernel Source

↓
Driver
Kernel

Source Code

↓
Driver
code

Drivers

↓
Static
in
tree

↓
Zimage

- ① Driver will always init
- ② Recompile full kernel in case of driver enhancement
- ③ fast in Initiation
- ④ slow in Booting

↓
Dynamic
Driver

↓
— .ko

- ① user have to initialize it (insmod)
- ② slow in init
- ③ Linux will boot faster

Makefile

Wednesday, June 10, 2026 7:16 PM

```
1 obj-m = podcast.o
2
3 KDIR = /lib/modules/$(shell uname -r)/build
4
5 all:
6     # run make file kernel
7     # run the target called modules
8     # the module exist in the current directory
9     make -C $(KDIR) M=$(shell pwd) modules
10
11 clean:
12     make -C $(KDIR) M=$(shell pwd) clean
13
```

initative Global variable

call another Makefile

```
###
# External module support.
# When building external modules the kernel used as basis is considered
# read-only, and no consistency checks are made and the make
# system is not used on the basis kernel. If updates are required
# in the basis kernel ordinary make commands (without #...) must be used.
```

→ M= → o/p Driver
↳ compile Driver

Driver.c → Driver.o → Driver.ko

Makefile flow

obj-m

↳ Dynamic Module

obj-y

↳ Static in Kernel

in Addition

you need to update
manually
kernel

Proc → debugging
 ↳ system behavior

Output we need to create
 file under proc

proc is a file system

#include "module.h" ↳ Data structure
 #include proc_fs.h ①

the Driver that we will
 implement doesn't
 depend on a device

Implementation will be based
 on the given functionality
 in proc_fs.h

How we will initialize the Driver?

- ① insmod — .ko
- ② init function

Deinitialized

- ① rmmod podcast
- ② Deinit function

```
/** Init function
 * @brief used in static in tree and out of tree [insmod]
 */
init static int my_module_init_function(void)
{
    printk("hello from init function \n");
    pdir = proc_create("podcast", 0666, NULL, &pops);
    return 0;
}
```

↳ status

↳ E-OK

module_init(==)

why static?
 eliminate other Drivers
 to initialize Driver

↳ why Linux make the init function in init section?

RAM Limited

↳ Save RAM

Remove Init Section the RAM

Why --exit section?

Static
↓
--exit
will not
be part
of your
Memory

Dynamic
↓
rmmod

MODULE_LICENSE
MODULE_AUTHOR → in order
your Driver
to compile

proc - file - operation

```
struct proc_ops {
    unsigned int proc_flags;
    int (*proc_open)(struct inode *, struct file *);
    ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
    /* mandatory unless nonseekable open() or equivalent is used */
    loff_t (*proc_lseek)(struct file *, loff_t, int);
    int (*proc_release)(struct inode *, struct file *);
    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
    long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif
    int (*proc_mmap)(struct file *, struct vm_area_struct *);
    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long, unsigned long, unsigned long, unsigned long);
} __randomize_layout;
```

open fd
read
write

file ← operation
↳ mmap

proc_create(name, mode, parent, proc_ops)

↓ ↓ ↓ ↓
file rw mode data
name == struc

Linux c

↳ Doesn't use std c library

In order to debug we need
to printf
↳ dmesg

proc - write

... error received

User → kernel

return size

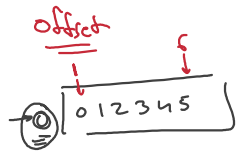
proc_read

return 0

kernel → user

echo hello > podcast

```
ssize_t podcast_write(struct file *fi, const char __user *user, size_t size, loff_t *off)
{
    // int ret = copy from user(arr, user, 4);
    // printk("Podcast is write %d\n", arr[0]);
    printk("Podcast is write\n");
    return size;
}
```



off = 0